



**UNIVERSIDADE ESTADUAL DA PARAÍBA
CENTRO DE CIÊNCIA E TECNOLOGIA - CCT
DEPARTAMENTO DE COMPUTAÇÃO
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**

ADRIELLY CARLA FERREIRA DE MELO - 2023208510013

PEDRO BRYAN ANDRADE DA COSTA - 2023208510003

Análise de Algoritmos de Busca e Ordenação

CAMPINA GRANDE - PB

2026

1. INTRODUÇÃO

Este trabalho tem como objetivo implementar e analisar algoritmos de busca e ordenação em Java, comparando o comportamento teórico esperado com os tempos obtidos na prática. A Proposta foi avaliar tanto estruturas baseadas em objetos da classe Estudante quanto arrays primitivos, permitindo comparar algoritmos clássicos de ordenação, buscas lineares e binárias, além da ordenação nativa da linguagem. Dessa forma, foi possível observar não apenas a complexidade assintótica de cada algoritmo, mas também o impacto do cenário de entrada no desempenho real.

A motivação principal da atividade foi relacionar teoria e prática. Em geral, algoritmos com pior complexidade podem até apresentar bom desempenho em alguns cenários pequenos ou favoráveis, enquanto algoritmos mais eficientes tendem a se destacar em entradas grandes. Assim, a análise realizada buscou identificar esses comportamentos, destacando em quais situações cada algoritmo é mais adequado.

2. METODOLOGIA

Os testes foram realizados em Java, utilizando dados criados para garantir repetibilidade e comparação justa entre os algoritmos. Foram considerados três cenários principais de entrada: aleatório, ordenado e inversamente ordenado. Esses cenários foram escolhidos porque permitem evidenciar melhor as diferenças entre os algoritmos, principalmente nos casos em que a estrutura da entrada influencia diretamente o tempo de execução.

Os dados foram gerados de forma controlada, permitindo que os algoritmos fossem executados sob as mesmas condições e com entradas equivalentes. Na ordenação, os testes foram aplicados sobre objetos da classe Estudante, o que aproxima a análise de um contexto mais realista, já que os algoritmos trabalham com registros e não apenas com valores numéricos. Também foi feita uma comparação entre o QuickSort implementado para inteiros e a ordenação nativa `Arrays.sort(int[])`, utilizada como referência prática de desempenho em arrays primitivos.

Para a ordenação de objetos do tipo Estudante, foram testados algoritmos quadráticos, como BubbleSort, BubbleSortOtimizado, SelectionSort, SelectionSortEstavel e InsertionSort, além de algoritmos de melhor desempenho

assintótico, como MergeSort, QuickSort, QuickSortShuffle, TimSortJava e CountingSort. Já na parte de busca, foram avaliados BuscaLinearIterativa, BuscaLinearRecursiva, BuscaBinariaIterativa, BuscaBinariaRecursiva e BuscaDuasPontas.

Os tamanhos de entrada foram definidos de acordo com a categoria dos algoritmos. Nos algoritmos quadráticos de ordenação, foram utilizados vetores com 1.000, 5.000 e 10.000 elementos, pois esses métodos crescem rapidamente em custo computacional. Nos algoritmos mais eficientes, foram testados tamanhos de 20.000, 100.000 e 500.000 elementos, permitindo observar o comportamento em entradas maiores. Na busca, também foram utilizados os tamanhos de 20.000, 100.000 e 500.000 elementos.

As medições foram feitas com repetição de execuções e cálculo do tempo médio em nanossegundos, buscando reduzir oscilações ocasionais do ambiente de execução. Antes das medições principais, foi realizada uma fase de aquecimento para minimizar efeitos iniciais da JVM. Além disso, em alguns casos, falhas como ****estouro de pilha**** em implementações recursivas foram mantidas como evidência de limitação prática de certos algoritmos, conforme esperado na análise.

Na busca, foram considerados os casos de elemento presente e elemento ausente. Esse critério foi adotado para observar o comportamento dos algoritmos tanto quando a chave é encontrada rapidamente quanto quando é necessário percorrer toda a estrutura até concluir que o elemento não está presente. Já na ordenação, a comparação entre objetos e inteiros foi importante para avaliar os algoritmos em cenários diferentes: os objetos simulam um uso mais próximo de aplicações reais, enquanto os inteiros permitem comparar o comportamento de uma implementação própria com a ordenação nativa da linguagem.

Os resultados foram consolidados em tabela e exportados para o arquivo CSV resultados-desempenho.csv, permitindo a organização dos dados e a posterior construção das análises comparativas.

3. RESULTADOS - TABELAS

3.1 Ordenação — algoritmos quadráticos (n até 10.000)

Tabela 1 — Tempo médio de execução dos algoritmos de ordenação (em ms)

Algoritmo	Entrada ordenada	Entrada ordenada	Entrada ordenada	Entrada aleatória	Entrada aleatória	Entrada aleatória	Entrada inversamente ordenada	Entrada inversamente ordenada	Entrada inversamente ordenada
	n=1.000	n=5.000	n=10.000	n=1.000	n=5.000	n=10.000	n=1.000	n=5.000	n=10.000
Bubble Sort	2,594	90,772	510,493	5,365	140,336	609,292	1,584	49,292	221,611
Bubble Sort Otimizado	0,014	0,072	0,160	5,189	140,894	630,111	1,615	51,749	236,252
Selection Sort	1,250	32,198	129,718	1,449	35,273	141,866	2,053	63,172	329,154
Selection Sort Estável	0,684	19,249	80,093	1,290	28,222	119,308	3,172	114,404	628,751
Insertion Sort	0,016	0,101	0,191	0,644	14,095	64,476	0,936	25,339	118,683

Os algoritmos quadráticos foram testados com entradas de 1.000, 5.000 e 10.000 elementos. O crescimento do tempo foi expressivo conforme o tamanho aumentou. Para $n = 10.000$ no cenário aleatório, o BubbleSort levou aproximadamente 609 ms, o SelectionSort 142 ms e o InsertionSort 64 ms. Já para $n = 1.000$, os mesmos algoritmos registraram 5,4 ms, 1,4 ms e 0,6 ms respectivamente, mostrando crescimento próximo ao esperado de $O(n^2)$.

O cenário ordenado revelou diferenças importantes entre os algoritmos. O BubbleSortOtimizado caiu de 5,188 ms (aleatório, $n = 1.000$) para apenas 0,014 ms no cenário ordenado, graças à parada antecipada. O InsertionSort teve comportamento semelhante: 0,191 ms no cenário ordenado contra 64,476 ms no aleatório para $n = 10.000$. Já o SelectionSort não apresentou essa melhora, mantendo tempos próximos independentemente do cenário, pois sempre percorre o vetor inteiro.

No cenário inversamente ordenado, o SelectionSortEstavel foi o mais prejudicado, atingindo 628 ms para $n = 10.000$, tempo superior até ao BubbleSort no mesmo cenário (221 ms). O InsertionSort, que se saiu bem no cenário ordenado, registrou 118 ms no inverso, confirmando sua sensibilidade ao grau de desordem da entrada.

3.2 Ordenação — algoritmos eficientes (n de 20.000 a 500.000)

Tabela 2 — Tempo médio de execução dos algoritmos eficientes (em ms)

Algoritmo	Entrada ordenada	Entrada ordenada	Entrada aleatória	Entrada aleatória	Entrada inversamente ordenada	Entrada inversamente ordenada
	n=100.000	n=500.000	n=100.000	n=500.000	n=100.000	n=500.000
Merge Sort	26,733	189,706	42,190	274,342	26,989	172,118
Tim Sort (Java)	3,245	20,996	39,272	232,677	4,497	28,597
Quick Sort	-	-	35,452	257,496	-	-
Quick Sort Shuffle	38,389	267,867	39,359	307,021	38,386	265,664
Counting Sort	5,687	46,038	36,461	209,873	5,704	49,578

"-" - Algoritmo registrou estouro de pilha(Quick Sort na tabela).

Para entradas maiores, os algoritmos de complexidade $O(n \log n)$ apresentaram tempos muito mais competitivos. Para $n = 500.000$ no cenário aleatório, o MergeSort levou 275 ms, o TimSortJava 221 ms, o QuickSort 234 ms e o CountingSort 209 ms.

O QuickSort clássico registrou ERROR (estouro de pilha) nos cenários ordenado e inversamente ordenado em todos os tamanhos testados a partir de $n = 20.000$, indicado nos dados csv como -1. Isso ocorre porque, sem embaralhamento prévio, o pivô fixo gera recursão degenerada nesses casos. O QuickSortShuffle, que embaralha os dados antes de ordenar, não apresentou esse problema e manteve tempos próximos ao MergeSort.

O TimSortJava se destacou nos cenários ordenado e inversamente ordenado, aproveitando padrões existentes nos dados. Para $n = 500.000$ ordenado, registrou apenas 21 ms, contra 190 ms do MergeSort no mesmo cenário. O CountingSort apresentou comportamento distinto dependendo do cenário. Nos cenários ordenado e inversamente ordenado, foi competitivo, registrando 5,687 ms e 5,704 ms para $n = 100.000$ respectivamente. No cenário aleatório, porém, seu desempenho ficou próximo ao do MergeSort, registrando 36,461 ms para $n = 100.000$ e 209,873 ms para $n = 500.000$. Isso indica que a implementação utilizada não se beneficiou da complexidade $O(n + k)$ de forma uniforme em todos os cenários, possivelmente pelo custo de construção do vetor auxiliar sobre entradas desordenadas.

3.3 Ordenação de inteiros — QuickSort vs Arrays.sort

A comparação entre o QuickSort implementado manualmente e o Arrays.sort nativo da linguagem mostrou desempenhos muito próximos. Para $n = 500.000$, o QuickSort levou 27 ms e o Arrays.sort 28,7 ms. Para $n = 20.000$, o Arrays.sort foi ligeiramente mais rápido (0,88 ms contra 1,12 ms). Em termos práticos, o Arrays.sort é preferível por ser mais robusto e já otimizado para diferentes cenários internamente.

3.4 Busca

Tabela 3 — Tempo médio de execução dos algoritmos de busca (em ns)

Tamanho do Vetor	n=20.000	n=100.000	n=500.000
BuscaLinear Iterativa (presente)	111.180	234.720	2.923.750
BuscaLinear Iterativa (ausente)	75.100	389.910	7.128.845
BuscaLinear Recursiva (presente)	-	-	-
BuscaLinear Recursiva (ausente)	-	-	-
BuscaBinária Iterativa (presente)	1.940	660	795
BuscaBinária Iterativa (ausente)	405	605	535
BuscaBinária Recursiva (presente)	2.345	230	300
BuscaBinária Recursiva (ausente)	145	275	1.755
BuscaDuas Pontas (presente)	210.745	379.625	5.442.295
BuscaDuas Pontas (ausente)	137.560	564.730	4.632.925

O — indica estouro de pilha em todos os tamanhos testados.

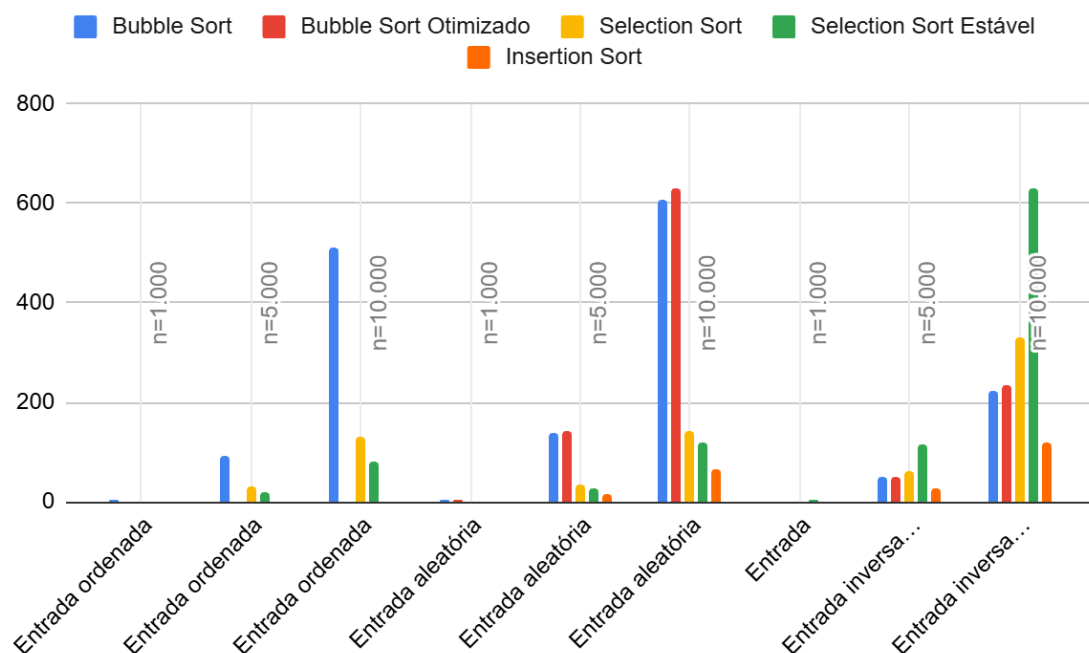
A diferença entre busca linear e busca binária ficou evidente nos dados. Para $n = 500.000$ com elemento presente, a BuscaLinearIterativa levou 2,9 ms enquanto a BuscaBinariaIterativa levou apenas 0,0008 ms. No cenário com elemento ausente e $n = 500.000$, a busca linear chegou a 7,1 ms enquanto a binária manteve 0,0005 ms, pois percorre o espaço de busca de forma logarítmica independentemente do resultado.

A BuscaDuasPontas, apesar de buscar simultaneamente pelos dois extremos do vetor, não trouxe vantagem prática relevante em relação à busca linear simples e chegou a ser mais lenta em alguns casos, como 5,4 ms para $n = 500.000$ com elemento presente.

A BuscaLinearRecursiva registrou falha em todos os cenários e tamanhos testados, por estouro de pilha de recursão. Esse resultado é importante: mesmo que o algoritmo seja conceitualmente correto, a implementação recursiva não é viável para entradas grandes em Java sem ajuste do tamanho da pilha da JVM.

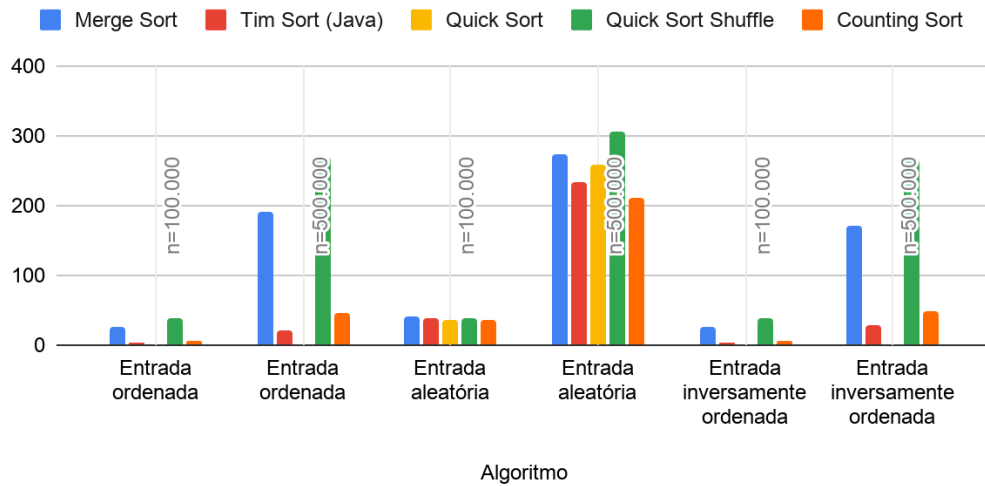
4. RESULTADOS EM FORMA DE GRÁFICOS

4.1 Ordenação — algoritmos quadráticos (n até 10.000)



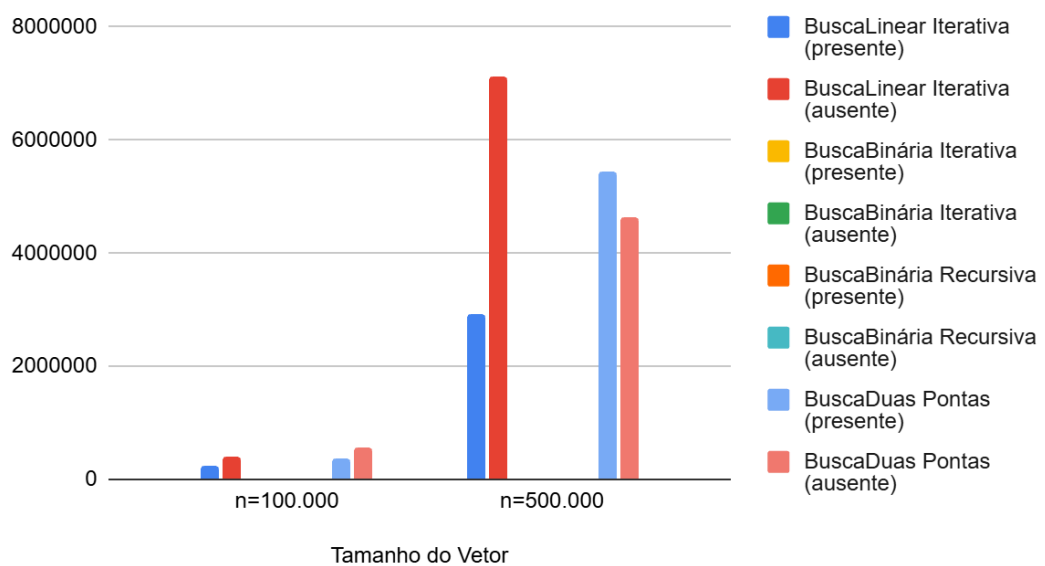
4.2 Ordenação — algoritmos eficientes (n de 20.000 a 500.000)

Entrada ordenada/n=100.000, Entrada ordenada/n=500.000,
Entrada aleatória/n=100.000, Entrada aleatória/n=500.000,



4.3 Busca

n=100.000 e n=500.000



5. ANÁLISE E DISCUSSÃO

Os resultados obtidos confirmaram, com dados concretos, o que a teoria prevê. Os algoritmos quadráticos cresceram de forma muito mais acentuada com o aumento da entrada: o BubbleSort passou de 5,4 ms para 609 ms ao ir de $n = 1.000$ para $n = 10.000$, um aumento de mais de 100 vezes para uma entrada apenas 10 vezes maior, compatível com $O(n^2)$. Em contraste, o MergeSort passou de 7,9 ms para 274 ms ao ir de $n = 20.000$ para $n = 500.000$, crescimento bem mais suave e consistente com $O(n \log n)$.

O cenário de entrada influenciou os resultados de forma significativa. O BubbleSortOtimizado e o InsertionSort foram os que mais se beneficiaram do cenário ordenado, reduzindo seus tempos em ordens de magnitude. Por outro lado, o SelectionSort foi indiferente ao cenário, pois sua lógica sempre percorre o vetor completo. O caso mais crítico foi o QuickSort clássico, que falhou completamente nos cenários ordenado e inversamente ordenado por estouro de pilha, demonstrando que a escolha do pivô é determinante para a robustez do algoritmo.

Quanto à estabilidade, algoritmos como InsertionSort, MergeSort, TimSortJava e BubbleSort preservam a ordem relativa de elementos iguais. O SelectionSort padrão não é estável, o que motivou a implementação do SelectionSortEstavel. Em aplicações reais com ordenação por múltiplos critérios, a estabilidade pode ser essencial.

Em relação ao uso de memória auxiliar, MergeSort e TimSortJava exigem espaço adicional proporcional ao tamanho da entrada, enquanto o QuickSort trabalha majoritariamente sobre o próprio vetor. O CountingSort exige um vetor auxiliar de tamanho proporcional ao intervalo de valores, o que pode ser inviável dependendo do domínio. Nos testes realizados, seu desempenho variou conforme o cenário: foi eficiente nas entradas ordenada e inversamente ordenada, mas no cenário aleatório ficou próximo ao MergeSort, sugerindo que o custo prático da implementação utilizada reduziu o ganho teórico esperado de $O(n + k)$.

As falhas da BuscaLinearRecursiva e do QuickSort em cenários específicos não devem ser tratadas apenas como se fossem erros de implementação, mas como evidências experimentais de limitações reais desses algoritmos. Elas mostram que a análise de algoritmos precisa ir além da complexidade assintótica, profundidade de recursão e comportamento em casos extremos.

Para uso em produção, a recomendação que os dados sustentam é clara: TimSortJava ou MergeSort para objetos, e Arrays.sort para primitivos. Algoritmos quadráticos são viáveis apenas para entradas pequenas, e o QuickSort clássico deve ser usado com embaralhamento prévio ou substituído pelo QuickSortShuffle.

6. CONCLUSÃO

A análise realizada permitiu verificar, na prática, os comportamentos esperados pela teoria da complexidade. Os algoritmos de ordenação quadrática apresentaram desempenho inferior em entradas maiores, enquanto os algoritmos de ordem $O(n \log n)$ mostraram melhor escalabilidade. Na busca, a busca binária confirmou sua superioridade em relação à busca linear quando aplicada sobre dados ordenados.

Além disso, o trabalho mostrou que o cenário de entrada influencia fortemente os resultados observados. Entradas ordenadas, aleatórias e inversamente ordenadas evidenciam vantagens e limitações diferentes em cada algoritmo. Também ficou claro que fatores como estabilidade, uso de memória auxiliar e robustez da implementação são decisivos na escolha de uma solução para uso real.

Portanto, conclui-se que a melhor estratégia não é apenas escolher o algoritmo com melhor complexidade teórica, mas sim aquele que oferece o melhor equilíbrio entre desempenho, memória, estabilidade e comportamento no cenário esperado de uso.